

Vyking DevOps Platform

Implementation and Design. A local Kubernetes platform delivered with Terraform, Argo CD (App of Apps), Helm, and GitOps.

14

Argo CD applications, 12 children in 7 sync waves

5

independent database protection layers

3

TLS certificates from a local CA

9 / 9

end to end verification groups passing

3

CI/CD workflows (lint, integration, publish)

1. Executive summary

This platform provisions a three node k3d cluster with Terraform, installs Argo CD, and uses an App of Apps GitOps model to deploy a MySQL database with automated backups, a frontend and backend application, and a set of platform capabilities: admission policy, TLS, observability (metrics, logs, cost), and alerting. Terraform owns only the bootstrap (Argo CD plus the two parent Applications); every other component is declared in Git and reconciled by Argo CD, so the GitOps configuration is itself under GitOps.

The objective was a platform that runs end to end from a single documented flow, where every component is declarative and version controlled, and where the operational concerns of a production system are represented even on a local cluster. The sections below map each requirement to its implementation, detail the extras and the design decisions behind them, show the concrete configuration (alert rules, resource specifications, policy code), and record the challenges encountered and the improvements that would follow.

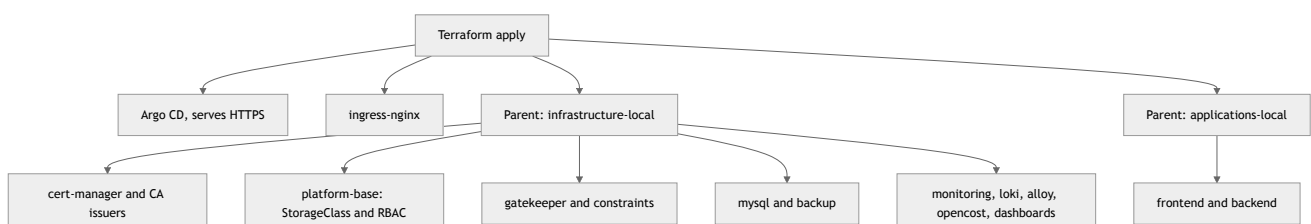
2. Repository structure

vyk-tech/ terraform/ modules/ argocd-apps.tf environments/ infrastructure/ argocd-apps/ base/ mysql/ backup/ gatekeeper/ gatekeeper-policies/ cert-manager/ monitoring/ applications/ scripts/ .github/workflows/	Bootstrap: Argo CD + ingress-nginx (Helm), two parent Apps Reusable modules: argocd, ingress-nginx, argocd-project, ar The two parent Applications (App of Apps) Per environment tfvars: local, staging, production Helm chart that renders the 12 child Applications (template StorageClass (Retain) + MySQL ServiceAccount/Role/RoleBindi Bitnami MySQL values (exporter, persistence, hardening) Backup PVC + every 5 minute CronJob OPA Gatekeeper DeletionProtection ConstraintTemplate The Constraints (which resources are protected) cert-manager values + local CA issuers (TLS) kube-prometheus-stack, Loki, Alloy, OpenCost values + dashb Custom Helm chart: frontend + backend (ingress, HPA, Networ init, create-cluster, build-images, create-secrets, teardown ci (lint/scan), integration (full k3d test), publish-images
---	--

Area	Responsibility
terraform/	Bootstrap only: install Argo CD and ingress-nginx, define the two AppProjects and parent Applications. Reusable modules; moved blocks keep refactors state neutral.
infrastructure/argocd-apps/	A Helm chart whose templates are the child Application manifests. The parent passes repoURL and targetRevision as parameters, so no branch name is committed.
infrastructure/<component>/	One directory per platform component: its Helm values or raw manifests.
applications/	The product workload: a custom Helm chart for the frontend and backend.
scripts/	The documented operational flow, from cluster creation to teardown.
.github/workflows/	Static gates, the full integration test, and image publishing.

3. Platform architecture

Terraform installs Argo CD (serving HTTPS) and ingress-nginx, then creates two AppProjects and two parent Applications. The infrastructure parent renders and applies its child Applications in sync wave order so that dependencies exist before their dependents.



Workloads run on the two tainted worker nodes (`role=worker:NoSchedule`); the control plane node is reserved for system components, mirroring a production topology. Resource names carry an environment suffix from the `environment` Terraform variable.

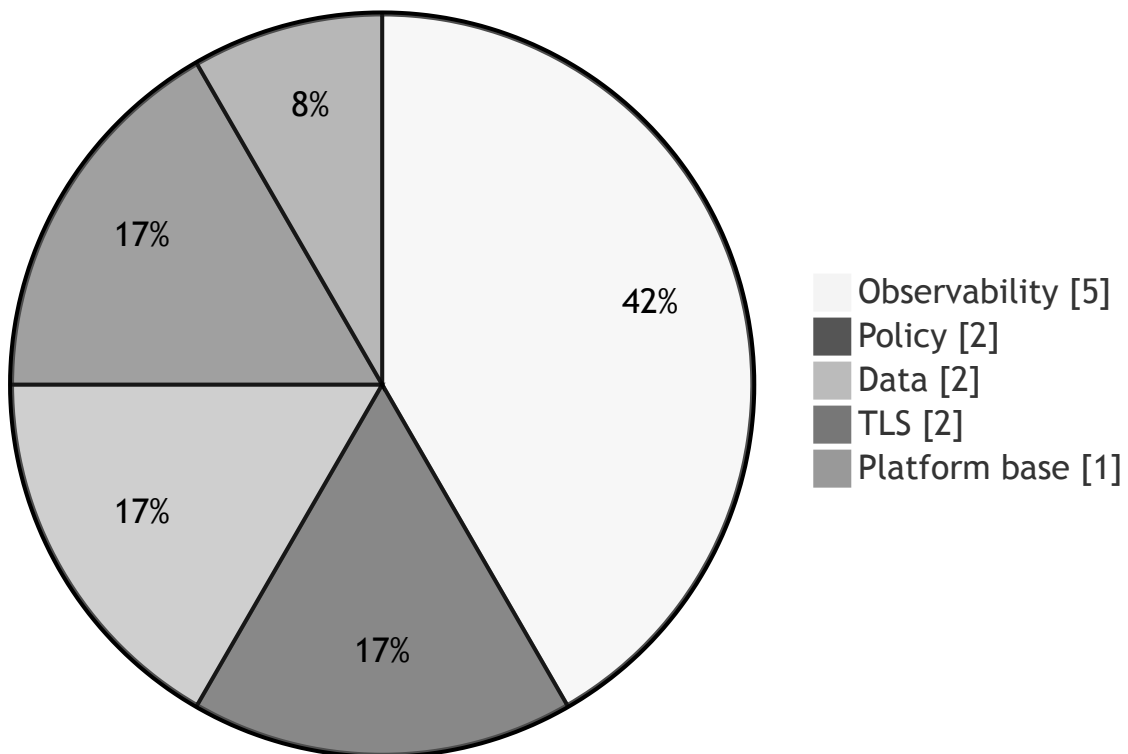
Sync wave ordering

Wave	Applications	Why this order
-2	cert-manager, platform-base	Install CRDs, the StorageClass, and RBAC before anything consumes them
-1	cert-manager-issuers	The CA chain needs the cert-manager CRDs from wave -2
0	gatekeeper, mysql, backup	Core data tier and the policy controller
1	gatekeeper-policies	Constraints need the ConstraintTemplate from wave 0
3	monitoring, loki	The metrics and logging stack (installs the ServiceMonitor CRD)
4	opencost, alloy, grafana-dashboards	Consumers of the wave 3 stack

4. Component inventory

Twelve child Applications across five categories, plus the two parents.

Child applications by category



Application	Category	Wave	Source	Purpose
cert-manager	TLS	-2	chart v1.18.2 + values	Controller, webhook, CRDs
cert-manager-issuers	TLS	-1	git directory	Self signed CA, CA issuer, Argo CD cert
platform-base	Base	-2	git directory	Retain StorageClass, MySQL RBAC
gatekeeper	Policy	0	chart 3.16.3 + template	OPA controller, ConstraintTemplate
gatekeeper-policies	Policy	1	git directory	DeletionProtection constraints
mysql	Data	0	chart 11.1.7 + values	Bitnami MySQL StatefulSet + exporter
backup	Data	0	git directory	Backup PVC + CronJob
monitoring	Observability	3	chart 85.3.3 + values	Prometheus, Grafana, Alertmanager
loki	Observability	3	chart 6.34.0 + values	Log store (single binary)
alloy	Observability	4	chart 1.8.2 + values	Log collector DaemonSet
opencost	Observability	4	chart 2.5.20 + values	Cost allocation
grafana-dashboards	Observability	4	git directory	Custom platform dashboard

5. Requirements traceability

Requirement	Implementation	Evidence
Infrastructure as Code	Terraform installs Argo CD + ingress-nginx and defines the parents and projects; reusable modules	<code>terraform/</code>
GitOps	Argo CD App of Apps; child apps are Git managed	<code>infrastructure/argocd-apps/</code>
Helm	MySQL, the app chart, and every add on are Helm based	values files per component
Database	Bitnami MySQL StatefulSet on a Retain StorageClass	<code>infrastructure/mysql/</code>
Backups	CronJob every 5 minutes, compressed dumps, self cleaning retention	<code>infrastructure/backup/</code>
Application	Frontend proxies to backend; backend reads and writes the database	<code>applications/</code>
Verification and teardown	Documented commands and a teardown script	<code>README.md</code> , <code>scripts/teardown.sh</code>

6. Observability

Three signals are collected and made queryable from a single Grafana instance.

Signal	Stack	Notes
Metrics	kube-prometheus-stack (Prometheus, Grafana, node-exporter, kube-state-metrics, operator)	MySQL exporter scraped via a ServiceMonitor; k3s in process components disabled to avoid permanently down targets
Logs	Loki (single binary, Retain PVC) + Grafana Alloy DaemonSet	Alloy tails the node level pod log directory, so logs persist in Loki after a pod crashes, restarts, or is deleted
Cost	OpenCost, reusing the platform Prometheus	Cost allocation by namespace; Kubecost is the documented upgrade

Dashboards include the kube-prometheus-stack defaults plus a custom "Vyking Platform Overview" board covering backend CPU and memory, MySQL connections, backup success, and backend logs.

7. Alerting

Alertmanager is enabled. Prometheus loads the kube-prometheus-stack default rules (pod crash looping, target down, node pressure, deployment replica mismatch, and more) plus the platform specific rules below, defined through `additionalPrometheusRulesMap`.

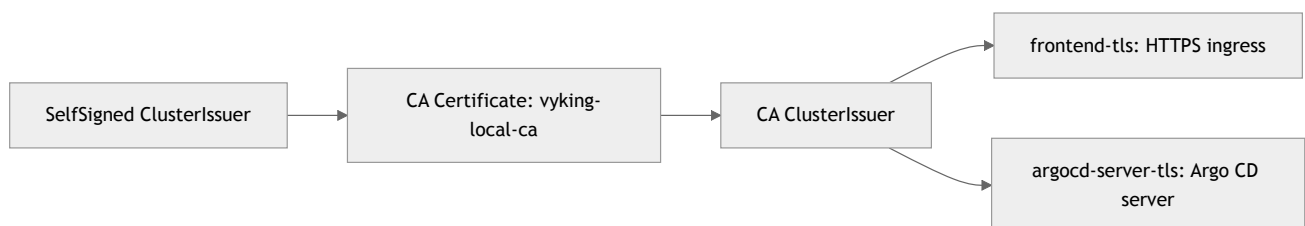
```
groups:
  - name: vyking.mysql
    rules:
      - alert: MySQLDown
        expr: mysql_up == 0
        for: 2m
        labels: { severity: critical }
        annotations:
          summary: "MySQL is down ({{ $labels.instance }})"
  - name: vyking.backup
    rules:
      - alert: MySQLBackupNotSucceeding
        expr: (time() - max(kube_job_status_completion_time{namespace="infrastructure"}))
        for: 5m
        labels: { severity: warning }
      - alert: MySQLBackupJobFailed
        expr: kube_job_status_failed{namespace="infrastructure"} > 0
        for: 1m
        labels: { severity: warning }
```

Alert	Condition	For	Severity	Meaning
MySQLDown	<code>mysql_up == 0</code>	2m	critical	The exporter reports the database is unreachable
MySQLBackupNotSucceeding	No completed backup Job in 30 minutes	5m	warning	The 5 minute CronJob has stopped succeeding
MySQLBackupJobFailed	<code>kube_job_status_failed > 0</code>	1m	warning	A backup Job ended in failure

8. Security and policy

TLS with a local certificate authority

A self signed bootstrap issuer mints a CA certificate; a CA ClusterIssuer then signs every leaf certificate, so they share one trust root. Production swaps the issuer for ACME or a corporate CA with no consumer changes.



The frontend ingress serves HTTPS and redirects plain HTTP; the Argo CD server runs with `server.insecure` disabled and a CA signed certificate that cert-manager writes to its serving secret.

Argo CD RBAC

Default deny, with three named roles. Enforcement was verified with `argocd admin settings rbac can`.

Role	applications get	applications sync	applications delete
platform-admin	yes	yes	yes
deployer	yes	yes	no
viewer	yes	no	no

Admission policy: deletion protection

Gatekeeper rejects deletion of protected resources at admission (StatefulSet, PVC, Secret, the MySQL Application). The policy reads the object under review on a DELETE (`oldObject`) and honors an explicit override annotation.

```
violation[{"msg": msg}] {
    input.review.operation == "DELETE"
    obj := input.review.oldObject
    obj.metadata.annotations["vyking.io/deletion-protected"] == "true"
    not obj.metadata.annotations["vyking.io/deletion-override"] == "true"
    msg := sprintf("Resource %s is deletion-protected", [obj.metadata.name])
}
```

Pod hardening and secrets

Every container runs non root with a read only root filesystem, all Linux capabilities dropped, and the RuntimeDefault seccomp profile. The database password is generated at setup with `openssl rand -base64 32` and stored only in a Kubernetes Secret; it is never hardcoded or committed. NetworkPolicies follow a default deny model, and the backend accepts traffic only from the frontend.

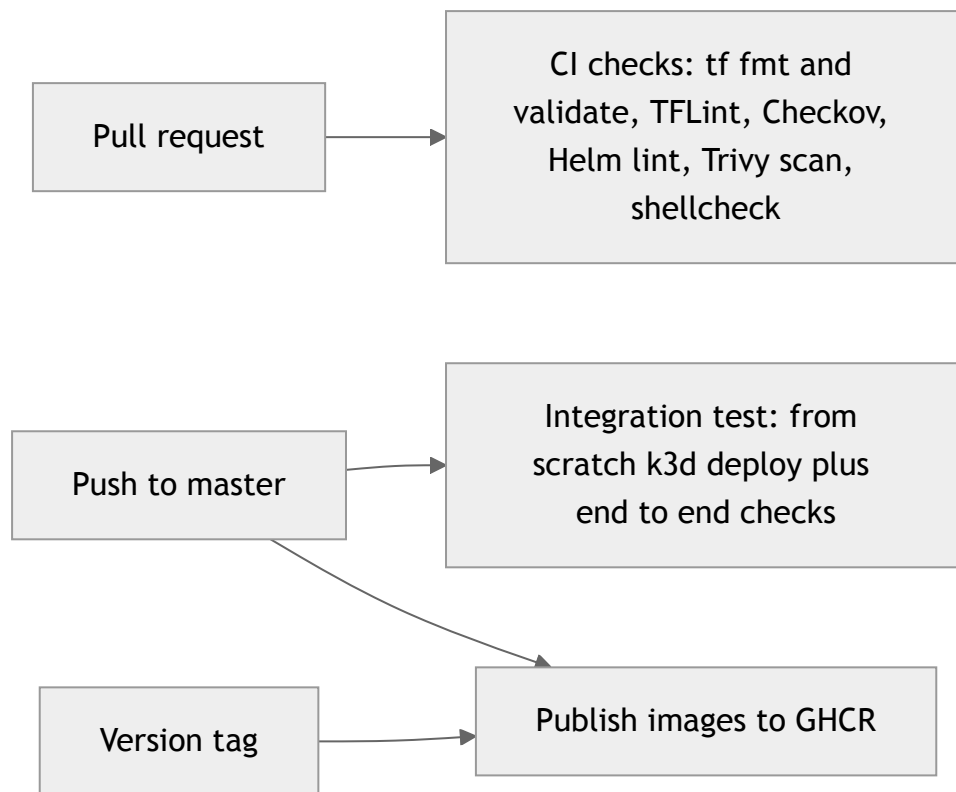
9. Resource management

Every workload sets requests and limits, tuned for a small cluster. Requests size the scheduler footprint; limits cap bursts.

Component	CPU req / limit	Memory req / limit
MySQL (server)	250m / 1000m	512Mi / 1Gi
MySQL exporter	10m / 100m	32Mi / 128Mi
Prometheus	150m / 750m	512Mi / 1Gi
Grafana	50m / 250m	256Mi / 512Mi
Prometheus operator	50m / 200m	128Mi / 256Mi
kube-state-metrics	20m / 100m	64Mi / 128Mi
node-exporter (per node)	20m / 100m	32Mi / 64Mi
Alertmanager	25m / 100m	64Mi / 128Mi
Loki	50m / 500m	128Mi / 512Mi
OpenCost (exporter + UI)	20m / 350m	96Mi / 384Mi
cert-manager (3 pods)	30m / 400m	160Mi / 320Mi
frontend / backend (each)	50m / 200m	64Mi / 128Mi

Scheduling: the data and application tiers are pinned to worker nodes with a `role=worker` toleration and nodeSelector. The monitoring workloads tolerate the worker taint so the scheduler can spread them off the single control plane node, which otherwise came under memory pressure on a constrained runner (see Challenges).

10. CI/CD and image delivery



Workflow	Trigger	Purpose
ci	pull request, push	Terraform fmt/validate, TFLint, Checkov, Helm lint and manifest validation, Trivy image scan, shellcheck
integration	push to master, manual	Build a fresh k3d cluster and run the end to end checks (convergence, worker placement, MySQL data and persistence, backend API, backups, ingress over HTTPS)
publish-images	push to master, version tags	Build and push the images to GHCR (<code>sha-<short></code> , <code>latest</code> , <code>semver</code>) using the built in token

Images are built locally and loaded with `k3d image import` for the cluster and the integration test (no registry dependency, fully offline). The publish workflow additionally pushes the same images to GHCR as registry artifacts. Pull based deployment from the registry is a documented improvement.

11. Key design decisions

Decision	Rationale
App of Apps	Terraform owns only the two parents; child apps are Git managed, so the GitOps config follows GitOps
Templated targetRevision	The App of Apps is a Helm chart; the parent passes the repo and branch as parameters, so no branch name is committed and a feature branch cannot leak its revision onto master at merge
Multi source Applications	Upstream charts are referenced directly and combined with repo values; chart upgrades need no vendoring
ServiceMonitor in the monitoring app	Its CRD ships at a later wave than MySQL; defining the monitor next to its CRD removes a cross wave dependency cycle
kubectl status in CI	The controller writes Application status to the CR; reading it with kubectl needs no Argo CD login over a port forward
cert-manager pinned to v1.18.2	Matches the cluster Kubernetes 1.29; newer CRDs use a field the 1.29 API server does not recognise
Generated secrets, outside Git	Credentials never enter Git; an operator such as External Secrets is the production path

12. Challenges faced

Challenge	Symptom	Root cause	Resolution
Image vulnerabilities	Trivy flagged Go standard library CVEs (one critical)	Backend image used an older Go toolchain	Raised the Go version until the scan was clean; Bitnami images pinned to the legacy archive (registry retired in 2025)
cert-manager vs cluster version	Deploy stalled at the first wave	cert-manager v1.20 CRDs use <code>selectableFields</code> (k8s 1.31+), unknown to the 1.29 API server, breaking server side apply	Pinned cert-manager to v1.18.2
App of Apps cross wave deadlock	MySQL never went Healthy; later waves never deployed	The MySQL app declared a ServiceMonitor whose CRD installs several waves later	Moved the ServiceMonitor into the monitoring app, next to its CRD
Gatekeeper not blocking	Deletes of protected resources succeeded	Constraints ran in warn mode	Switched to deny; teardown lifts the webhook before destroy
Stale targetRevision after merge	Master could not resolve the child apps source	A merged feature branch baked its own branch name into the committed manifests, and the branch was later deleted	Templated targetRevision so the branch is a deploy time parameter, never committed
API server under load in CI	Verification steps lost the API connection	The 2 vCPU runner starved the k3s API server at peak load	Added a stabilization gate plus retries on watch based calls
Monitoring on the control plane	A monitoring pod was evicted; the app went Degraded	Workloads had no worker toleration and all landed on the control plane node	Added tolerations so the scheduler spreads them

13. Possible improvements

Area	Current state	Improvement
Alerting interface	Prometheus rules plus Alertmanager	Grafana native alerting for one interface across metrics and logs
Alert coverage	DB and backup rules plus k8s defaults	Certificate expiry, application error rate, database saturation
Notifications	Default Alertmanager receiver	Slack, email, or PagerDuty receivers
Certificates	Local self signed CA	ACME issuer or a corporate CA
Secrets	Pre created Kubernetes Secrets	External Secrets, SOPS, or Vault with encryption at rest
Image delivery	Built and imported; also published to GHCR	Pull based deployment from the registry with an immutable tag strategy and a pull secret
Database	Single StatefulSet	Managed or replicated database for production
Cost	OpenCost	Kubecost for a richer interface and cloud billing
Delivery strategy	Rolling updates	Argo Rollouts for canary or blue green

14. Verification status

A from scratch GitOps deploy reached all Applications Synced and Healthy. The table summarizes the end to end checks.

Group	Result
Cluster and GitOps (nodes, taints, all apps Synced/Healthy)	Pass
Database (seed data in appdb, schema correct)	Pass
Backups (CronJob, timestamped dumps persisting on the PVC)	Pass
Application (health endpoints, full CRUD)	Pass
Ingress (HTTPS 200, CA signed cert, HTTP redirect)	Pass
Observability (Prometheus targets, Grafana dashboards, Loki logs, OpenCost)	Pass
Policy (Gatekeeper constraints, deny on a protected resource)	Pass
Persistence (data survives pod and StatefulSet deletion)	Pass
Infrastructure as Code (terraform validate, fmt)	Pass

A separate review confirmed no leaked secrets, least privilege RBAC, NetworkPolicies in place, no privileged pods, and Gatekeeper enforcing deletion protection in deny mode.

Part II: Production-readiness enhancements

The sections above describe the platform as required and as deployed. The sections below catalog, with concrete configuration, the engineering added on top of the basic brief to make the platform behave like a production system: high availability and zero downtime, supply chain and runtime security, network segmentation, a managed certificate authority, and observability wired for correctness.

0

maxUnavailable on rolling updates (zero downtime)

2 to 6

backend autoscaling range (CPU 70%, memory 80%)

3

NetworkPolicies enforcing a default deny model

12 to 0

backend image high and critical CVEs after the Go rebuild

10 yr / 1 yr

CA root and leaf certificate lifetimes

15. Enhancements at a glance

Category	Enhancement	What it adds
Availability	Rolling updates with maxUnavailable 0	Deployments roll without dropping a single request
Availability	HorizontalPodAutoscaler (backend)	Scales 2 to 6 on CPU and memory; absorbs load spikes
Availability	PodDisruptionBudgets	Keeps a replica available during node drains and upgrades
Availability	topologySpreadConstraints	Spreads replicas across worker nodes; one node loss is survivable
Availability	Split liveness and readiness probes	A database blip removes a pod from rotation instead of restarting it
Security	Distroless, non-root, read-only root filesystem	No shell, no package manager, no write access, minimal attack surface
Security	Go 1.26 image rebuild	Cleared one critical and eleven high standard library CVEs
Security	Trivy gate in CI (fail on high and critical)	A vulnerable image cannot be merged
Security	NetworkPolicies (default deny)	Lateral movement is blocked; only declared flows are allowed
Security	automountServiceAccountToken false	Pods carry no API token they do not need
Security	Local certificate authority (cert-manager)	Every endpoint served over TLS from one trust root
Security	Generated secrets, never committed	Credentials minted at setup, stored only as Kubernetes Secrets
Security	Gatekeeper deletion protection (deny)	Admission rejects deletion of stateful resources
Observability	Metrics, logs, cost, dashboards, alerts	A full operations view from a single Grafana
Delivery	GitOps App of Apps with templated revision	The whole platform reconciles from Git, branch agnostic
Delivery	GHCR image publishing	Images available as registry artifacts with a tag strategy

16. High availability and zero-downtime deployments

Every product workload runs at least two replicas and is configured so that no single update, node drain, or load spike causes downtime.

Rolling updates. Both Deployments use a `maxSurge: 1, maxUnavailable: 0` strategy, so Kubernetes starts a new pod and waits for it to pass readiness before it removes an old one. No request is ever routed

to a pod that is not ready.

Autoscaling. The backend has a HorizontalPodAutoscaler (autoscaling/v2) on both CPU and memory, with scale up and scale down behavior tuned to avoid thrashing. When autoscaling is enabled the Deployment omits a static `replicas` value, so the HPA and Argo CD self heal do not fight over the replica count.

Disruption budgets and spreading. PodDisruptionBudgets keep at least one replica available during voluntary disruptions, and topologySpreadConstraints distribute replicas across worker nodes.

Setting	Backend	Frontend
Replicas (baseline)	2 (HPA managed)	2
maxSurge / maxUnavailable	1 / 0	1 / 0
HPA min / max	2 / 6	not autoscaled
HPA targets	CPU 70%, memory 80%	not applicable
HPA scale up	2 pods per 30s, 30s window	not applicable
HPA scale down	1 pod per 60s, 300s window	not applicable
PodDisruptionBudget	minAvailable 1	minAvailable 1
Spread	maxSkew 1 by hostname, DoNotSchedule	maxSkew 1 by hostname, DoNotSchedule
terminationGracePeriod	30s	30s

A `checksum/config` annotation on the pod template ties each rollout to the content of its ConfigMap, so a configuration change triggers a controlled rolling restart automatically.

17. Health checking strategy

The three probe types are deliberately separated rather than all pointed at one endpoint, which is a common cause of restart storms.

Probe	Backend path	Frontend path	Purpose
startupProbe	/healthz	/healthz	Allows up to 90s (30 x 3s) for cold start before liveness applies
livenessProbe	/healthz	/healthz	Unconditional 200; restarts only a genuinely hung process
readinessProbe	/health	/healthz	Backend /health pings the database; a DB blip de-registers the pod rather than killing it

The key decision is the backend split: liveness hits `/healthz` (always 200 while the process is alive), while readiness hits `/health` (which pings MySQL). A transient database problem therefore takes the pod out of the Service endpoints until it recovers, instead of triggering a restart loop that would deepen

the outage. Backend timings: liveness initialDelay 10s, period 10s, timeout 5s, failureThreshold 3; readiness initialDelay 5s, period 5s, timeout 3s, failureThreshold 3; startup failureThreshold 30, period 3s.

18. Container image and supply-chain security

Minimal, non-root images. The backend is a multi-stage build: it compiles in `golang:1.26-alpine` and ships from `gcr.io/distroless/static:nonroot`. The final image has no shell and no package manager, runs as the baked in `nonroot` user (uid 65532), and contains only a single static binary (`CGO_ENABLED=0` , stripped with `-ldflags="-w -s"`). An `ARG TARGETARCH` lets BuildKit produce the correct architecture for Apple Silicon or CI without a hardcoded value. The frontend runs `nginx:1.27-alpine` as user 101 on a non privileged port.

Vulnerability remediation. A Trivy scan flagged Go standard library CVEs carried by the backend build version, including one critical and eleven high (for example CVE-2025-68121 in crypto/tls and CVE-2025-22871 in net/http). Rebuilding on Go 1.26 cleared them, and the build comment documents keeping the Go version current.

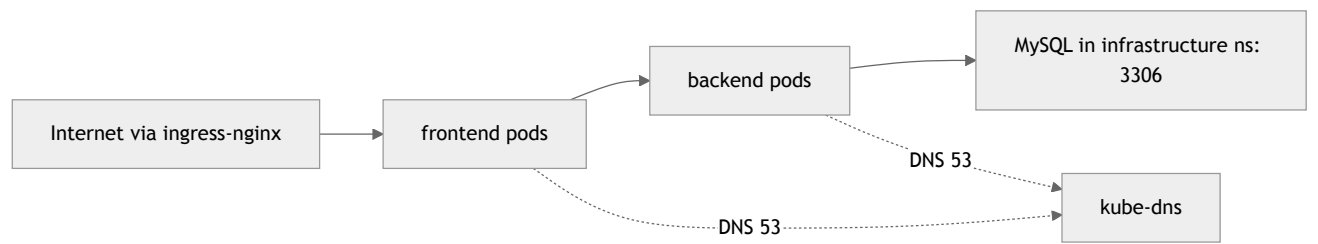
Scanning as a merge gate. CI runs Trivy on the backend image with `severity: HIGH,CRITICAL` and `exit-code: 1` , so a vulnerable image fails the build. Checkov scans the Terraform, Helm, and Kubernetes manifests, and an OPA conftest step checks policy; the Terraform scan runs with `soft_fail: false` .

Runtime hardening, applied to every pod.

Control	Setting
runAsNonRoot	true (uid 1000 backend, 101 frontend)
readOnlyRootFilesystem	true, with emptyDir mounts for /tmp and nginx cache and run
allowPrivilegeEscalation	false
capabilities	drop ALL
seccompProfile	RuntimeDefault (pod and container level)
automountServiceAccountToken	false
fsGroup	set at pod level for correct volume ownership

19. Network segmentation

Three NetworkPolicies implement a default deny model in the applications namespace: nothing communicates unless a policy explicitly allows it.



- **default-deny-all** selects every pod and denies all ingress and egress.
- **frontend-policy** allows ingress on the frontend port from anywhere (it is the public entry point) and egress only to the backend and DNS.
- **backend-policy** allows ingress only from frontend pods and egress only to MySQL (port 3306 in the infrastructure namespace) and DNS.

The result is that a compromised frontend cannot reach the database directly, and nothing in the namespace can open arbitrary outbound connections.

20. Certificate authority and TLS detail

cert-manager runs a three step local PKI so every certificate shares one trust root, and the issuer can be swapped for ACME or a corporate CA in production without touching any consumer.

Resource	Kind	Detail
vyking-selfsigned-issuer	ClusterIssuer (SelfSigned)	Bootstrap only; used to mint the CA
vyking-ca	Certificate (isCA)	CN vyking-local-ca, ECDSA P-256, 10 year duration, secret vyking-ca-key-pair
vyking-ca-issuer	ClusterIssuer (CA)	Signs every leaf certificate
frontend-tls	Certificate (leaf)	Served by ingress-nginx for vyking.local
argocd-server-tls	Certificate (leaf)	1 year, SANs include argocd.vyking.local; Argo CD hot reloads it

Argo CD runs with `server.insecure` removed, so the server serves HTTPS using the CA signed `argocd-server-tls` secret instead of its default self signed certificate, and reloads automatically when cert-manager rotates it.

21. Observability configuration depth

Beyond enabling the stacks, the monitoring layer is wired for correctness on this cluster:

- **Metrics discovery** is opened to all namespaces (`serviceMonitorSelectorNilUsesHelmValues: false`), and the MySQL exporter ServiceMonitor is defined in the monitoring app next to its CRD, which removes a cross wave ordering problem.
- **k3s in-process control plane** ServiceMonitors are disabled so Prometheus does not display permanently down targets for components that do not expose separate endpoints.
- **Dashboards** combine the kube-prometheus-stack defaults (node, cluster, namespace, and pod CPU and memory) with a sidecar that auto loads any ConfigMap labeled `grafana_dashboard` , which is how the custom Vyking Platform Overview board is delivered; a datasource sidecar wires Loki in as a log source.
- **Alerting** uses Alertmanager together with the platform rules in section 7, loaded through `additionalPrometheusRulesMap` .